

bulk_register_aips ❖❖❖ Manuale

Direzione ICT, Università❖❖ degli Studi di Milano

Luglio 2026

Contents

Manuale — bulk_register_aips	1
1. Scopo	1
2. Come funziona, passo per passo	2
3. Il path “a quadranti” e il contratto del current_path	3
4. Evento PREMIS di compressione e pointer file	3
5. Fallback di identificazione via METS	4
6. Cross-check di coerenza UUID	4
7. Report JSON	4
8. Configurazione (.env)	5
9. Dopo la registrazione: indicizzazione nel Dashboard	5
10. Procedura completa per l’ingestion di AIP orfani	6
11. Troubleshooting	7
12. Note di sicurezza	8

Manuale — bulk_register_aips

1. Scopo

Questo strumento risolve un problema specifico: hai una cartella con AIP (Archival Information Package) che esistono fisicamente sul filesystem, ma che la Storage Service di Archivemantica non conosce — cioè non hanno un record nella sua tabella dei package (Package). Senza quel record, l’AIP:

- non compare tra i risultati di ricerca “Archival Storage” del Dashboard,
- non può essere scaricato, verificato (fixity) o cancellato tramite l’interfaccia o l’API della Storage Service,
- non è considerato “gestito” dal sistema di preservazione.

Casi d’uso tipici:

- **AIP ereditati** da un’altra installazione Archivemantica di cui si è perso il database e resta solo la cartella dello store (già organizzata a quadranti);
- **AIP prodotti da strumenti terzi** (a3m, DART3) depositati in una cartella piatta;
- **recuperi da backup** dove i file esistono ma il record è andato perso.

Lo script automatizza il confronto tra ciò che è **fisicamente presente** su disco e ciò che è **logicamente noto** alla Storage Service, e registra la differenza (gli AIP “orfani”) con una chiamata API per ciascuno.

La registrazione nella Storage Service è solo una parte del processo: per rendere gli AIP visibili anche nella ricerca del Dashboard serve poi la reindicizzazione (capitolo 9). La procedura

operativa completa, dall’inizio alla fine, è descritta nel capitolo 10.

2. Come funziona, passo per passo

2.1 Inventario fisico

`find_local_aips()` scansiona ricorsivamente `--scan-dir` con `os.walk`, quindi trova i file a qualunque livello di annidamento, indipendentemente da come sono organizzate le sotto-cartelle.

Per ogni file trovato, cerca di determinarne l’UUID in due modi:

1. **Dal nome del file**, se rispetta il pattern `<nome>-<uuid>.<estensione>` (es. `mio-dataset-3fa85f64-5717-4562-b3fc-2c963f66afa6.7z`). Questo è il caso più comune per AIP prodotti normalmente da Archivematica.
2. **Dal METS interno**, se il nome del file non contiene un UUID riconoscibile ma l’estensione è un formato di archivio supportato (`.7z`, `.tar.gz`, `.tar.bz2`, `.tar`, `.zip`). Utile per AIP prodotti da strumenti come **a3m** o **DART3**, o per file rinominati manualmente. Vedi capitolo 5.

2.2 Inventario logico

`get_registered_aip_uuids()` interroga (con paginazione) `GET /api/v2/file/?package_type=AIP` sulla Storage Service e raccoglie tutti gli UUID già registrati come package di tipo AIP. Nota: anche i package in stato `DELETED` restano nel database con il loro UUID e vengono considerati “già registrati” — non è possibile ri-registrare un UUID già presente, in qualunque stato sia.

2.3 Differenza

`find_orphan_aips()` confronta i due inventari e distingue:

- **orfani “sicuri”**: presenti su disco, non registrati, nessuna incoerenza rilevata tra nome file e METS interno;
- **conflitti**: presenti su disco, non registrati, MA con UUID nel nome file diverso da quello nel METS interno (capitolo 6). Questi **non** vengono registrati automaticamente.

2.4 Registrazione

Per ogni AIP orfano “sicuro”, `register_aip()` esegue una `POST /api/v2/file/` alla Storage Service. Il payload include:

Campo	Valore	Note
<code>uuid</code>	UUID dell’AIP	estratto dal filename o dal METS, mai generato
<code>origin_location / origin_path</code>	dove il file si trova ORA	path relativo alla location, com’è su disco
<code>current_location</code>	location di destinazione	<code>--target-location-uuid</code>
<code>current_path</code>	SOLO il nome del file	vedi capitolo 3: lo sharding lo aggiunge la SS
<code>package_type</code>	"AIP"	
<code>size</code>	dimensione in byte	
<code>origin_pipeline</code>	URI della pipeline	obbligatorio , da <code>PIPELINE_UUID</code> in <code>.env</code>
<code>events</code>	evento PREMIS di compressione	obbligatorio per AIP compressi , vedi capitolo 4

La POST non è una semplice scrittura nel database: la Storage Service esegue un vero **store** (store_aip), che comprende uno spostamento via rsync da origin a destinazione (no-op se coincidono) e la creazione del pointer file. Non esiste una modalità “registra soltanto, non toccare i file”.

Attenzione: se la POST fallisce a metà (es. HTTP 500), può restare nel database un **record parziale** — che in certi casi (es. origin_pipeline nullo) rompe perfino le GET di elenco per tutti i client con errore 400. Dopo ogni run fallito, controllare la Django admin della Storage Service (<http://<host>:8000/admin/>, sezione Packages) ed eliminare i record spuri prima di ritentare.

3. Il path “a quadranti” e il contratto del current_path

Archivematica organizza gli AIP dentro una location secondo uno schema deterministico basato sull’UUID, per evitare cartelle con decine di migliaia di file:

```
<4hex>/<4hex>/<4hex>/<4hex>/<4hex>/<4hex>/<4hex>/<nome-file>
```

Gli otto blocchi da 4 caratteri sono **tutti** i 32 caratteri esadecimali dell’UUID (senza trattini); il file sta direttamente nell’ultimo quadrante, senza cartella intermedia con l’UUID completo. Esempio, per l’UUID 125b6250-333d-477b-9f35-c9ebc1467b45:

```
125b/6250/333d/477b/9f35/c9eb/c146/7b45/test_002-125b6250-333d-477b-9f35-c9ebc1467b45.7z
```

(Schema verificato empiricamente sul current_path restituito da GET /api/v2/file/<uuid>/ della Storage Service UNIMI, luglio 2026.)

Punto fondamentale, verificato nel sorgente della Storage Service (package.py, store_aip):

```
uuid_path = utils.uuid_to_path(self.uuid)
self.current_path = os.path.join(uuid_path, self.current_path)
```

La Storage Service **prepende sempre da sé** il path a quadranti al current_path ricevuto. Di conseguenza:

- nel payload va passato **solo il nome del file** come current_path;
- passare un path già shardato produce percorsi **raddoppiati** (quadranti/quadranti/file) — con spostamento fisico dei file nella posizione sbagliata;
- lo schema dipende solo dall’UUID, quindi un AIP proveniente da un’altra istanza Archivematica, già a quadranti, finisce esattamente dove già si trova: lo spostamento rsync è un no-op e i file non vengono toccati.

Lo script calcola comunque il path a quadranti atteso (compute_quadrant_path()) per **prevedere e loggare** se lo store comporterà uno spostamento fisico reale (CON spostamento fisico, caso cartella piatta) o un no-op (senza spostamento, caso già a quadranti). Questa previsione è relativa a --scan-dir: perché sia accurata, --scan-dir deve puntare esattamente alla **radice** della location (la cartella che contiene direttamente i quadranti di primo livello).

4. Evento PREMIS di compressione e pointer file

Per gli AIP **compressi** (file, non directory) la Storage Service crea durante lo store un **pointer file**: un METS esterno con i dettagli PREMIS della compressione, necessario per le future operazioni di decompressione e verifica. Per crearlo le serve un evento PREMIS di tipo compression nel campo events della POST. Senza, lo store fallisce con HTTP 500 e l’errore (nel log della Storage Service):

This AIP needs a pointer file, however the Archivematica pipeline did not create one and it also did not provide the compression event needed for the Storage Service to create one.

Lo script costruisce automaticamente questo evento (`build_compression_event()`), replicando la struttura che `store_aip.py` di Archivematica invia normalmente. Il programma e l'algoritmo di compressione vengono dedotti dal tipo di archivio (`detect_compression()`):

- `.7z` → programma `7z`, algoritmo letto dall'archivio con `7z l -slt` (LZMA/LZMA2 → `lzma`, BZip2 → `bzip2`, Copy → `copy`; default `lzma`);
- `.tar.gz/.tgz` → `gzip`; `.tar.bz2/.tbz2` → `pbzip2`;
- `.tar` e `.zip` → best effort (non sono formati di compressione nativi di Archivematica).

La nota dell'evento dichiara esplicitamente che si tratta di una **registrazione retrospettiva**: descrive la compressione osservata, non l'operazione di compressione originale (onestà archivistica verso chi leggerà il pointer file in futuro).

5. Fallback di identificazione via METS

Quando il nome del file non contiene un UUID riconoscibile (es. AIP da a3m/DART3 con convenzioni di naming diverse, o file rinominati), lo script apre l'archivio **senza estrarlo interamente** e cerca l'UUID autoritativo nel METS:

1. Cerca un'entry con nome `METS.<uuid>.xml` — è il caso più comune, e basta leggere l'elenco dei contenuti dell'archivio (veloce).
2. Se non trovata, cerca un'entry che sembra un METS (`mets*.xml`), ne legge il contenuto e cerca l'UUID nell'attributo `OBJID` dell'elemento `<mets:mets>`.

Formati supportati nativamente in Python: `.zip`, `.tar`, `.tar.gz`, `.tar.bz2`. Per `.7z` è necessario il binario `7z` (pacchetto `p7zip-full`) nel PATH: se non è disponibile, viene stampato un avviso e quell'AIP viene ignorato dalla scansione (a meno che il nome del file non contenga già un UUID valido).

Questo comportamento si può disattivare con `--no-mets-fallback` (più veloce su collezioni molto grandi, ma ignora gli AIP con nomi non standard e disabilita anche il cross-check descritto sotto).

6. Cross-check di coerenza UUID

Anche quando il nome del file **contiene già** un UUID valido, lo script apre comunque l'archivio (se `--no-mets-fallback` non è passato) e confronta quell'UUID con quello scritto nel METS interno. Il motivo: il nome del file potrebbe essere stato alterato manualmente, o l'AIP potrebbe essere una copia rinominata di un altro — e l'UUID autoritativo è quello nel METS, non quello nel nome.

Quando i due non coincidono, l'AIP viene marcato come **conflitto** e **non viene registrato automaticamente**: compare in una sezione separata dell'output e del report JSON (`conflicts`), per una verifica manuale caso per caso (file rinominato per errore, reingest, copia di test...).

7. Report JSON

Al termine dell'esecuzione viene scritto un file di report (default `bulk_register_report.json`):

```
{
  "scan_dir": "...",
  "dry_run": true,
  "results": {
```

```

    "<uuid>": {"success": true, "status_code": 201, "will_move": false, "uuid_source": "filenam
  },
  "conflicts": {
    "<uuid-dal-nome-file>": {"rel_path": "...", "filename_uuid": "..."}
  }
}

```

8. Configurazione (.env)

Vedi .env.example. Variabili:

- SS_URL: URL base della Storage Service. Attenzione al protocollo e all'host: alcune installazioni espongono la porta 8000 in HTTPS (con certificato self-signed → SSL_VERIFY=false), altre in HTTP semplice. Un errore SSL: WRONG_VERSION_NUMBER indica che si sta usando https:// verso un server HTTP; un 400 immediato su ogni richiesta può indicare un host non incluso negli ALLOWED_HOSTS di Django.
- SS_USER, SS_API_KEY: credenziali API della Storage Service.
- SSL_VERIFY: false per certificati self-signed.
- TARGET_LOCATION_UUID: UUID della location AIP Storage di destinazione (Storage Service > Locations).
- PIPELINE_UUID: UUID della pipeline a cui attribuire gli AIP. **Obbligatorio** (vedi 2.4). Si ricava con GET /api/v2/pipeline/ o dal Dashboard (Administration > General).

Nota per chi usa export \$(grep -v '^#' .env | xargs) nella shell: lo script legge il .env con setdefault, quindi eventuali variabili già esportate nella sessione **hanno la precedenza** sul file. Dopo aver modificato il .env, verificare con echo \$SS_URL che non ci siano valori stantii esportati (unset SS_URL per pulirli).

9. Dopo la registrazione: indicizzazione nel Dashboard

La registrazione nella Storage Service **non** rende gli AIP cercabili nella scheda "Archival storage" del Dashboard: quella ricerca usa indici Elasticsearch separati (aips e aipfiles), da aggiornare con un management command sul server del Dashboard.

Su installazioni recenti (layout src/archivematica/, virtualenv unificato) il comando richiede il caricamento dell'environment file:

```

sudo -u archivematica bash -c " \
  set -a -e
  source /etc/default/archivematica-dashboard || \
    (echo 'Environment file not found'; exit 1)
  /usr/share/archivematica/virtualenvs/archivematica/bin/python \
    -m archivematica.dashboard.manage \
    rebuild_aip_index_from_storage_service -u <UUID>
"

```

Opzioni utili: -u/--uuid per un solo AIP, --delete per rimuovere prima le voci con quell'UUID, --delete-all per rigenerare l'intero indice, --pipeline per una pipeline diversa da quella corrente.

Due avvertenze importanti, verificate sul campo:

1. **Il messaggio "Successfully indexed" può mentire.** La funzione interna di indicizzazione cattura tutte le eccezioni, le scrive solo sul logger e restituisce un codice d'errore che il comando **non controlla** prima di stampare il messaggio di successo (bug noto, presente a luglio 2026). Verificare sempre l'esito reale interrogando Elasticsearch:

```
curl -s 'http://127.0.0.1:9200/aips/_search?pretty' \
-H 'Content-Type: application/json' \
-d '{"query":{"match":{"uuid":"<UUID>"}}}'
```

2. **Un AIP senza file indicizzabili è invisibile nella UI.** La pagina Archival storage costruisce l'elenco partendo dall'indice aipfiles (i documenti dei singoli file) e aggregando da lì gli UUID degli AIP. Un AIP il cui METS non contiene un fileSec con file nei gruppi original/metadata viene indicizzato con Files indexed: 0: risulta presente nell'indice aips e interrogabile via API, ma **non compare mai** nella ricerca del Dashboard. Con AIP reali (METS completi) il problema non si pone; può presentarsi con AIP sintetici, di test, o prodotti da tool terzi con METS minimali.

10. Procedura completa per l'ingestion di AIP orfani

Questa è la sequenza operativa completa, testata end-to-end (luglio 2026), per portare un insieme di AIP orfani dalla cartella su disco alla piena visibilità nel Dashboard.

10.1 Preparazione

1. **Posiziona i file in un percorso visibile alla Storage Service.** Mai in /tmp: il servizio gira tipicamente con systemd PrivateTmp=true (verificabile con `systemctl cat archivematica-storage-service | grep PrivateTmp`) e ha un /tmp privato — i file nel /tmp di sistema sono invisibili al processo, e rsync fallisce con “No such file or directory” su file che per te esistono. Un percorso adatto: sotto /var/archivematica/.
2. **Verifica i permessi:** l'utente del servizio (di norma archivematica) deve poter leggere i file e scrivere nella cartella (lo store fa un move rsync). Se lavori con il tuo utente, aggiungilo al gruppo archivematica oppure sistema la ownership con chown.
3. **Crea o individua la location AIP Storage di destinazione** nella Storage Service (purpose AS), annotane l'UUID e verifica che il suo path corrisponda alla cartella dei file. Per una registrazione “in place” di AIP già a quadranti, la radice della location deve essere la cartella che contiene direttamente i quadranti di primo livello.
4. **Compila il .env:** SS_URL, SS_USER, SS_API_KEY, SSL_VERIFY, TARGET_LOCATION_UUID, PIPELINE_UUID (capitolo 8).
5. **Verifica lo stato di partenza** della Storage Service:

```
curl -s -o /dev/null -w "%{http_code}\n" \
"<SS_URL>/api/v2/file/?package_type=AIP&limit=1" \
-H "Authorization: ApiKey <utente>:<api-key>"
```

Deve rispondere 200. Un 400 sistematico indica un record corrotto nel database (vedi 2.4) o un problema di host/protocollo (capitolo 8).

10.2 Dry-run

```
python3 bulk_register_aips.py \
--scan-dir /percorso/della/location \
--target-location-uuid <UUID_LOCATION> \
--dry-run
```

Controlla nell'output:

- il numero di AIP trovati e orfani corrisponde alle attese;
- per ogni AIP: current_path inviato è il **solo nome del file** e il path finale (calcolato dalla SS) è coerente;

- gli AIP già a quadranti risultano senza spostamento; quelli in cartella piatta CON spostamento fisico;
- eventuali **conflitti** UUID elencati a parte: vanno risolti a mano prima di procedere (capitolo 6).

10.3 Registrazione reale

Stesso comando senza `--dry-run`. Per ogni [0K] la Storage Service ha creato il record, l'eventuale spostamento fisico e il pointer file.

In caso di [ERROR] HTTP 500: leggere il log della Storage Service (`/var/log/archivematica/storage-service/storage_service.log`) per la causa reale, e **pulire gli eventuali record parziali** dalla Django admin prima di ritentare (2.4).

10.4 Verifica della registrazione

```
curl -s "<SS_URL>/api/v2/file/<uuid-registrato>/" \
  -H "Authorization: ApiKey <utente>:<api-key>" | python3 -m json.tool
```

Controllare: `status: UPLOADED`, `current_path` a 8 quadranti (senza raddoppi), `origin_pipeline` valorizzato, `size` corretta. Verificare anche sul filesystem che i file siano nella posizione attesa.

10.5 Indicizzazione e verifica nel Dashboard

Per ciascun UUID registrato, lanciare il comando di reindicizzazione (capitolo 9), controllando nell'output la riga `Files indexed: <n>` (per AIP reali deve essere `> 0`) e verificando l'effettiva presenza del documento in Elasticsearch. Infine, controllare la comparsa degli AIP nella scheda Archival storage del Dashboard.

10.6 In caso di problemi: ripartire puliti

Se un ciclo va storto, prima di ritentare:

1. Django admin → Packages → eliminare i record dei run falliti;
2. verificare che la GET di elenco risponda 200;
3. se erano già stati indicizzati documenti in Elasticsearch per UUID che non esistono più:

```
curl -s -X POST 'http://127.0.0.1:9200/aips/_delete_by_query' \
  -H 'Content-Type: application/json' \
  -d '{"query":{"terms":{"uuid":["<uuid1>","<uuid2>"]}}}'
```

4. se dei file sono stati spostati in posizioni sbagliate, riportarli nella posizione corretta (o ripartire da una copia pulita).

11. Troubleshooting

Sintomo	Causa / rimedio
SSL: <code>WRONG_VERSION_NUMBER</code>	SS_URL usa <code>https://</code> verso un server HTTP: correggere il protocollo
HTTP 400 su ogni richiesta, anche GET	host non in <code>ALLOWED_HOSTS</code> di Django, oppure un record Package corrotto nel DB (es. <code>origin_pipeline</code> nullo da un run fallito): eliminarlo dalla Django admin

Sintomo	Causa / rimedio
HTTP 400 sulla POST: <code>origin_pipeline ... doesn't allow a null value</code>	manca <code>PIPELINE_UUID</code> nel <code>.env</code> (o versione vecchia dello script)
HTTP 500, log: <code>rsync No such file or directory</code> su file esistenti	i file sono in <code>/tmp</code> e il servizio ha <code>PrivateTmp=true</code> : spostarli (es. sotto <code>/var/archivematica/</code>)
HTTP 500, log: <code>This AIP needs a pointer file...</code>	manca l'evento PREMIS di compressione nel payload (versione vecchia dello script)
File finiti in percorsi con quadranti raddoppiati	è stato inviato un <code>current_path</code> già shardato (versione vecchia dello script): la SS prepende sempre i quadranti da sé
[WARN] comando <code>'7z'</code> non trovato	installare <code>p7zip-full</code> sul server dove gira lo script
Un AIP scompare dalla scansione	nome file non conforme e fallback METS senza esito: archivio corrotto o METS con nome non standard
Un AIP finisce sempre in <code>conflicts</code>	il METS riporta davvero un UUID diverso dal nome file: verificare manualmente quale sia quello corretto
<code>will_move=True</code> inatteso su AIP già a quadranti	<code>--scan-dir</code> non punta alla radice esatta della struttura (capitolo 3)
"Successfully indexed" ma l'AIP non è in Elasticsearch	fallimento silenzioso del comando di reindicizzazione (bug noto): rilanciare e verificare con curl su ES (capitolo 9)
AIP in Elasticsearch (aips) ma invisibile nella UI	<code>file_count: 0</code> — il METS non ha file indicizzabili: la UI parte dall'indice <code>aipfiles</code> (capitolo 9)

12. Note di sicurezza

- Testare sempre `--dry-run` prima di un'esecuzione reale.
- Testare su un piccolo sottoinsieme in ambiente non di produzione prima di un run massivo, specialmente se `will_move=True` (spostamento fisico reale dei file).
- Un run fallito può lasciare record parziali nel database: verificare e pulire prima di ritentare (10.6).
- Il file `.env` contiene una API key: non va mai committato (verificare che sia escluso dal `.gitignore` del repo).
- L'UUID di un AIP non si inventa e non si cambia: è quello assegnato alla creazione, scritto nel METS. Registrare un AIP con un UUID diverso da quello interno crea un'incoerenza permanente (per questo esistono il cross-check e la quarantena dei conflitti).